Lab Handout 2-F

Feature Extraction Datapath and Feature Block Random Access Memory (BRAM)

Capstone — Module 2: Field-Programmable Gate Array (FPGA) Register-Transfer Level (RTL) Design and Signal Ingestion

1 Objective

Transform each validated frame into a fixed 32-byte feature vector and store it in a dual-port BRAM indexed by Beacon ID. By the end of this handout the pipeline will accumulate features per station across multiple frames and expose a read port for the Module 3 host interface. The bring-up `seg7_driver` from LH1-H is removed in this handout (the seven-segment display is replaced by feature-vector readout in Module 3).

2 Prerequisites

- Lab Handout 2-E (parser emits `frame_valid` only for double-Cyclic Redundancy Check (CRC)-clean frames).

3 Feature Vector Layout

The feature vector is 32 bytes, aligned to one BRAM row. The layout matches `tab:feature_vector` in the Module 2 lesson plan and `tab:featvec` in the final report — do not reorder fields.

Offset	Field	Size	Source
0	Beacon ID	1 B	radio packet byte 2 (frame offset 12)
1	Modulation Kind	1 B	(radio packet byte 6 $\gg$ 2)
2	Spreading Factor	1 B	radio packet byte 7 (frame offset 17)
3	`pkt_type`	1 B	radio packet byte 6 & `0x03`
4–5	RSSI (latest)	2 B	metadata RSSI sign-extended to 16 bits
6–7	SNR (latest)	2 B	metadata SNR sign-extended to 16 bits
8–11	Frame count (for this ID)	4 B	internal counter
12–15	Last arrival timestamp	4 B	metadata RX timestamp (ms)
16–19	Inter-arrival time (ticks)	4 B	current_ts − stored_ts
20–23	RSSI delta	4 B	current_rssi − stored_rssi (sign-extended)
24–27	Priority score	4 B	see Step 3 below
28	`target_id_if_pause`	1 B	radio payload byte 0 if PAUSE; `0x00` otherwise
29	`pause_duration_units`	1 B	radio payload byte 1 if PAUSE; `0x00` otherwise
30–31	Reserved / padding	2 B	—

4 Procedure

4.1 Step 1 — Retire the `seg7_driver` from the active design

The seven-segment display is no longer driven from this point on; the Module 3 host interface (and the `LED(15:8)` byte selector introduced in Step 4 below) replaces it. You can either remove `seg7_driver.vhd` from the project entirely, or leave it on disk as a reference and simply stop instantiating it. Either approach works because Step 4's rewrite of `ingest_top.vhd` drops the `u_seg` instance and forces the seven-segment outputs to inactive levels (`CA`–`CG`, `DP`, `AN` all high).

1. **Option A (recommended for a clean tree):** Sources panel → right-click `seg7_driver.vhd` → **Remove File from Project** → **tick** "Also delete the file from disk."
2. **Option B (keep the file as a reference):** leave `seg7_driver.vhd` where it is. Once you save the new `ingest_top.vhd` in Step 4, the file will simply be unused by synthesis; Vivado will flag it with a yellow "unused module" note rather than an error. Module 2 grading does not penalize either choice.

4.2 Step 2 — Create `feature_bram.vhd`

Sources panel: + → **Add or create design sources** → **Create File**, name `feature_bram`. Replace contents with the listing below.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity feature_bram is
    port (
        clk      : in  std_logic;
        rst_n    : in  std_logic;
        -- Write port (driven by feature_extract).
        wr_en    : in  std_logic;
        wr_addr  : in  std_logic_vector(7 downto 0);
        wr_data  : in  std_logic_vector(255 downto 0);
        -- Read port (driven by feature_extract for read-modify-write,
        -- and by the Module 3 host interface for readback).
        rd_addr_a : in  std_logic_vector(7 downto 0);
        rd_data_a : out std_logic_vector(255 downto 0);
        rd_addr_b : in  std_logic_vector(7 downto 0);
        rd_data_b : out std_logic_vector(255 downto 0)
    );
end feature_bram;

architecture rtl of feature_bram is
    type mem_t is array (0 to 255) of std_logic_vector(255 downto 0);
    signal mem : mem_t := (others => (others => '0'));
begin
    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                mem       <= (others => (others => '0'));
                rd_data_a <= (others => '0');
```

```
                rd_data_b <= (others => '0');
            else
                if wr_en = '1' then
                    mem(to_integer(unsigned(wr_addr))) <= wr_data;
                end if;
                rd_data_a <= mem(to_integer(unsigned(rd_addr_a)));
                rd_data_b <= mem(to_integer(unsigned(rd_addr_b)));
            end if;
        end if;
    end process;
end rtl;
```

Listing 1: `feature_bram.vhd` — 256-row × 256-bit dual-port BRAM. Inferred as two 36-Kb tiles by Vivado. The synchronous reset on `mem` and the registered read outputs is required for clean Vivado xsim startup; without it, simulation reads of an as-yet-unwritten address can return stale signal values from the testbench harness rather than the 0-initialized memory contents.

4.3 Step 3 — Create `feature_extract.vhd`

The priority score is a fixed integer combination so the implementation is deterministic and matches the Python reference in LH2-G:

$$\text{priority} = 4 \cdot \text{rssi_signed} + 1 \cdot \text{snr_signed} + \text{frame_count}$$

All terms are sign-extended to 32 bits before addition. The weights (4, 1, 1) come from the Module 2 lesson plan Section 4 and are intentionally simple — Module 4 (the AI inference layer) replaces this scoring with a learned model.

Create `feature_extract`, replace its contents with the listing below.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity feature_extract is
    port (
        clk          : in  std_logic;
        rst_n        : in  std_logic;
        -- From pkt_parser
        frame_valid  : in  std_logic;
        metadata     : in  std_logic_vector(63 downto 0);
        payload      : in  std_logic_vector(255 downto 0);
        feat_busy    : out std_logic;
        -- BRAM ports
        bram_wr_en   : out std_logic;
        bram_wr_addr : out std_logic_vector(7 downto 0);
        bram_wr_data : out std_logic_vector(255 downto 0);
        bram_rd_addr : out std_logic_vector(7 downto 0);
        bram_rd_data : in  std_logic_vector(255 downto 0)
    );
end feature_extract;

architecture rtl of feature_extract is
    type state_t is (S_IDLE, S_READ, S_COMPUTE, S_WRITE);
```

```
    signal state : state_t := S_IDLE;

    -- Latched inputs (held stable through the three-cycle update).
    signal meta_l    : std_logic_vector(63 downto 0)  := (others => '0');
    signal payload_l : std_logic_vector(255 downto 0) := (others => '0');

    -- Helpers: byte slice from the packed payload (byte 0 in LSB).
    function pbyte (p : std_logic_vector(255 downto 0); idx : integer)
        return std_logic_vector is
    begin
        return p((idx * 8 + 7) downto (idx * 8));
    end function;

    function mbyte (m : std_logic_vector(63 downto 0); idx : integer)
        return std_logic_vector is
    begin
        return m((idx * 8 + 7) downto (idx * 8));
    end function;
begin
    feat_busy <= '0' when state = S_IDLE else '1';

    process (clk)
        variable beacon       : std_logic_vector(7 downto 0);
        variable mod_code     : std_logic_vector(7 downto 0);
        variable pkt_type     : std_logic_vector(7 downto 0);
        variable mod_kind     : std_logic_vector(7 downto 0);
        variable sf           : std_logic_vector(7 downto 0);
        variable rssi_s       : signed(15 downto 0);
        variable snr_s        : signed(15 downto 0);
        variable cur_ts       : unsigned(31 downto 0);
        variable old_ts       : unsigned(31 downto 0);
        variable old_rssi     : signed(15 downto 0);
        variable old_count    : unsigned(31 downto 0);
        variable new_count    : unsigned(31 downto 0);
        variable inter_arr    : unsigned(31 downto 0);
        variable rssi_delta   : signed(31 downto 0);
        variable priority     : signed(31 downto 0);
        variable target_id    : std_logic_vector(7 downto 0);
        variable pause_dur    : std_logic_vector(7 downto 0);
        variable vec          : std_logic_vector(255 downto 0);
    begin
        if rising_edge(clk) then
            bram_wr_en <= '0';

            if rst_n = '0' then
                state <= S_IDLE;
            else
                case state is
                    when S_IDLE =>
                        if frame_valid = '1' then
                            meta_l       <= metadata;
                            payload_l    <= payload;
                            bram_rd_addr <= pbyte(payload, 2);   -- Beacon
                                ID
```

```
                    state        <= S_READ;
                end if;
            when S_READ =>
                -- BRAM read latency = 1 cycle; read data appears next cycle.
                state <= S_COMPUTE;
            when S_COMPUTE =>
                beacon    := pbyte(payload_l, 2);
                mod_code  := pbyte(payload_l, 6);
                sf        := pbyte(payload_l, 7);
                pkt_type  := "000000" & mod_code(1 downto 0);
                mod_kind  := "00" & mod_code(7 downto 2);

                rssi_s    := resize(signed(mbyte(meta_l, 4)), 16);
                snr_s     := resize(signed(mbyte(meta_l, 5)), 16);
                cur_ts    := unsigned(mbyte(meta_l, 3) & mbyte(meta_l, 2)
                                      & mbyte(meta_l, 1) & mbyte(meta_l, 0));

                -- Decode previously-stored fields from bram_rd_data.
                old_ts    := unsigned(bram_rd_data(127 downto 96));   -- offsets 12..15
                old_count := unsigned(bram_rd_data(95 downto 64));   -- offsets 8..11
                old_rssi  := signed(bram_rd_data(47 downto 32));   -- offsets 4..5

                new_count := old_count + 1;
                inter_arr := cur_ts - old_ts;
                rssi_delta := resize(rssi_s, 32) - resize(old_rssi, 32);
                priority   := shift_left(resize(rssi_s, 32), 2)
                              + resize(snr_s, 32)
                              + signed(resize(new_count, 32));

                if pkt_type = x"02" then
                    target_id := pbyte(payload_l, 13);   -- LH1-G buf[13]
                    pause_dur := pbyte(payload_l, 14);   -- LH1-G buf[14]
                else
                    target_id := (others => '0');
                    pause_dur := (others => '0');
                end if;

                -- Pack the 32-byte feature vector. Byte 0 in LSB.
                vec := (others => '0');
                vec(7 downto 0)     := beacon;                                   -- 0
                vec(15 downto 8)    := mod_kind;                                 -- 1
                vec(23 downto 16)   := sf;
```

```
                                -- 2
                            vec(31 downto 24)   := pkt_type;
                                                                    -- 3
                            vec(47 downto 32)   := std_logic_vector(rssi_s);
                                                        -- 4-5
                            vec(63 downto 48)   := std_logic_vector(snr_s);
                                                         -- 6-7
                            vec(95 downto 64)   := std_logic_vector(new_count)
                                ;                       -- 8-11
                            vec(127 downto 96)  := std_logic_vector(cur_ts);
                                                        -- 12-15
                            vec(159 downto 128) := std_logic_vector(inter_arr)
                                ;                       -- 16-19
                            vec(191 downto 160) := std_logic_vector(rssi_delta
                                );                      -- 20-23
                            vec(223 downto 192) := std_logic_vector(priority);
                                                        -- 24-27
                            vec(231 downto 224) := target_id;
                                                                    -- 28
                            vec(239 downto 232) := pause_dur;
                                                                    -- 29
                            -- 30-31 left zero.

                            bram_wr_addr <= beacon;
                            bram_wr_data <= vec;
                            bram_wr_en   <= '1';
                            state        <= S_WRITE;
                        when S_WRITE =>
                            state <= S_IDLE;
                    end case;
                end if;
            end if;
        end process;
end rtl;
```

Listing 2: `feature_extract.vhd` — read-modify-write of one BRAM row per `frame_valid`. Three-cycle latency: address out on cycle 0, BRAM read appears on cycle 1, write back on cycle 2.

4.4 Step 4 — Update `ingest_top.vhd`

The Module 2 LH2-D `ingest_top` drove `seg7_driver`. With `seg7_driver` removed, retire the seven-segment ports (drive them to safe inactive levels — `seg` all-high, `an` all-high) and instantiate `feature_bram` and `feature_extract`. The Module 3 host readback path is exposed on hardware via the on-board slide switches and high LEDs: `SW(7:0)` selects which Beacon-ID row, `SW(12:8)` selects which of the 32 bytes inside that row to display, and `LED(15:8)` shows the selected byte. The full 256-bit BRAM read-port output is held on an internal signal (`host_rd_data_int`) that the LH2-G end-to-end testbench accesses through a VHDL-2008 external name — it is not a top-level port, so no constraint is needed for it.

Replace `ingest_top.vhd` contents with the listing below.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```vhdl
use IEEE.NUMERIC_STD.ALL;

entity ingest_top is
    port (
        CLK100MHZ    : in  std_logic;
        CPU_RESETN   : in  std_logic;
        uart_rx_pin  : in  std_logic;                        -- PMOD JA pin 1 (C17)
        UART_RXD_OUT : out std_logic;                        -- onboard USB-UART (LH2-A)
        SW           : in  std_logic_vector(15 downto 0);
        CA, CB, CC, CD, CE, CF, CG : out std_logic;
        DP           : out std_logic;
        AN           : out std_logic_vector(7 downto 0);
        LED          : out std_logic_vector(15 downto 0)
    );
end ingest_top;

architecture rtl of ingest_top is
    signal rst_n      : std_logic;

    signal rx_data    : std_logic_vector(7 downto 0);
    signal rx_strobe  : std_logic;

    signal fifo_data  : std_logic_vector(7 downto 0);
    signal fifo_rd_en : std_logic;
    signal fifo_empty : std_logic;
    signal fifo_full  : std_logic;

    signal metadata     : std_logic_vector(63 downto 0);
    signal payload      : std_logic_vector(255 downto 0);
    signal rx_crc       : std_logic_vector(7 downto 0);
    signal frame_valid  : std_logic;
    signal frame_reject : std_logic;

    signal feat_busy    : std_logic;
    signal bram_wr_en   : std_logic;
    signal bram_wr_addr : std_logic_vector(7 downto 0);
    signal bram_wr_data : std_logic_vector(255 downto 0);
    signal bram_rd_addr : std_logic_vector(7 downto 0);
    signal bram_rd_data : std_logic_vector(255 downto 0);

    -- Module 3 host readback path. SW(7:0) selects the BRAM row;
    -- SW(12:8) selects which of 32 bytes within that row goes to LED (15:8).
    -- The LH2-G testbench reaches host_rd_data_int via VHDL-2008 external name.
    signal host_rd_data_int : std_logic_vector(255 downto 0);
    signal host_rd_byte     : std_logic_vector(7 downto 0);
begin
    rst_n <= CPU_RESETN;

    u_uart : entity work.uart_rx
        port map (clk => CLK100MHZ, rst_n => rst_n,
```

```
                    rx => uart_rx_pin,
                    rx_data => rx_data, rx_strobe => rx_strobe);

    u_fifo : entity work.byte_fifo
        port map (clk => CLK100MHZ, rst_n => rst_n,
                    wr_en => rx_strobe, wr_data => rx_data,
                    rd_en => fifo_rd_en, rd_data => fifo_data,
                    empty => fifo_empty, full => fifo_full);

    u_parser : entity work.pkt_parser
        port map (clk => CLK100MHZ, rst_n => rst_n,
                    fifo_empty => fifo_empty, fifo_data => fifo_data,
                    fifo_rd_en => fifo_rd_en,
                    feat_busy => feat_busy,
                    metadata => metadata, payload => payload, rx_crc =>
                        rx_crc,
                    frame_valid => frame_valid, frame_reject => frame_reject
                        );

    u_feat : entity work.feature_extract
        port map (clk => CLK100MHZ, rst_n => rst_n,
                    frame_valid => frame_valid,
                    metadata => metadata, payload => payload,
                    feat_busy => feat_busy,
                    bram_wr_en => bram_wr_en, bram_wr_addr => bram_wr_addr,
                    bram_wr_data => bram_wr_data,
                    bram_rd_addr => bram_rd_addr, bram_rd_data =>
                        bram_rd_data);

    u_bram : entity work.feature_bram
        port map (clk => CLK100MHZ, rst_n => rst_n,
                    wr_en => bram_wr_en, wr_addr => bram_wr_addr, wr_data =>
                         bram_wr_data,
                    rd_addr_a => bram_rd_addr, rd_data_a => bram_rd_data,
                    rd_addr_b => SW(7 downto 0), rd_data_b =>
                        host_rd_data_int);

    -- Byte selector inside the 32-byte feature row.
    process (host_rd_data_int, SW)
        variable idx : integer range 0 to 31;
    begin
        idx := to_integer(unsigned(SW(12 downto 8)));
        host_rd_byte <= host_rd_data_int((idx * 8 + 7) downto (idx * 8));
    end process;

    -- Seven-segment display retired in this handout; force outputs
        inactive.
    -- (seg7_driver.vhd may remain on disk per Step 1 Option B; it is no
        longer
    -- instantiated from here.)
    CA <= '1'; CB <= '1'; CC <= '1'; CD <= '1';
    CE <= '1'; CF <= '1'; CG <= '1'; DP <= '1';
    AN <= (others => '1');

```

```
    LED(0)              <= rx_strobe;
    LED(1)              <= frame_valid;
    LED(2)              <= fifo_empty;
    LED(3)              <= fifo_full;
    LED(4)              <= frame_reject;
    LED(5)              <= feat_busy;
    LED(7 downto 6)     <= (others => '0');
    LED(15 downto 8)    <= host_rd_byte;

    -- Passthrough: forward the raw PMOD JA1 stream out the onboard USB-
       UART
    -- (added in LH2-A so the host can capture .bin files via the same USB
    -- cable that programs the board). Both ends are 115200 8N1.
    UART_RXD_OUT <= uart_rx_pin;
end rtl;
```

Listing 3: `ingest_top.vhd` (LH2-F revision) — adds `u_feat` and `u_bram`; `feat_busy` feeds back into `pkt_parser`; seven-segment outputs forced inactive; `SW` drives a row/byte selector that displays one byte of the feature row on `LED(15:8)`.

4.5 Step 4b — Update the XDC

The new `SW` input port requires constraints. Open the project's `Nexys-A7-100T-Master.xdc` and uncomment *all sixteen* `SW[0]` through `SW[15]` lines (search for pin `J15` for `SW[0]`). The seven-segment, anode, LED, clock, reset, and `uart_rx_pin` constraints from LH1-H Step 3 remain unchanged. Save the XDC and re-run synthesis; the previously-failing DRC error about unconstrained ports should clear.

Why `host_rd_data` is no longer a top-level port

The 256-bit feature-vector readback path is too wide for the Nexys A7's available I/O, and exposing it as a top-level output would force the student to constrain 256 unused pins. Instead, the design routes one selectable byte to `LED(15:8)` for hardware visibility, and the Module 3 host interface (Module 3 Session 9) replaces this stub with a real AXI-Lite read port to the soft-core processor. The LH2-G end-to-end testbench reaches the full internal 256-bit signal through a VHDL-2008 external name, so simulation coverage of all 256 bits is preserved.

4.6 Step 5 — Feature extractor testbench

Add a simulation source `tb_feature_extract`. The testbench builds a tiny harness around `feature_extract` and `feature_bram`, injects six frames (3×DATA from ID `0x01`, 2×ACK from ID `0x02`, 1×PAUSE from ID `0xFF` addressed to Target A with duration 0x05), and reads back rows `0x01`, `0x02`, and `0xFF` through the host port.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_feature_extract is
end tb_feature_extract;

```

[Figure placeholder]

`LH2F_feature_datapath.png`

Feature extraction datapath. The BRAM read port feeds the compute block; the write port commits the updated vector on the following cycle.

Figure 1: Feature extraction datapath. The BRAM read port feeds the compute block; the write port commits the updated vector on the following cycle.

```
architecture sim of tb_feature_extract is
    constant CLK_PERIOD : time := 10 ns;

    signal clk     : std_logic := '0';
    signal rst_n   : std_logic := '0';

    signal frame_valid : std_logic := '0';
    signal metadata    : std_logic_vector(63 downto 0)  := (others => '0');
    signal payload     : std_logic_vector(255 downto 0) := (others => '0');
    signal feat_busy   : std_logic;

    signal wr_en   : std_logic;
    signal wr_addr : std_logic_vector(7 downto 0);
    signal wr_data : std_logic_vector(255 downto 0);
    signal rd_a    : std_logic_vector(7 downto 0);
    signal rd_da   : std_logic_vector(255 downto 0);
    signal rd_b    : std_logic_vector(7 downto 0) := (others => '0');
    signal rd_db   : std_logic_vector(255 downto 0);

    procedure inject_frame (signal fv : out std_logic;
                            signal m  : out std_logic_vector(63 downto 0);
                            signal p  : out std_logic_vector(255 downto 0);
                            beacon  : std_logic_vector(7 downto 0);
                            mod_b6  : std_logic_vector(7 downto 0);
                            ts      : unsigned(31 downto 0);
                            rssi    : signed(7 downto 0);
                            snr     : signed(7 downto 0);
                            tgt     : std_logic_vector(7 downto 0);
                            dur     : std_logic_vector(7 downto 0)) is
        variable mv : std_logic_vector(63 downto 0)  := (others => '0');
        variable pv : std_logic_vector(255 downto 0) := (others => '0');
    begin
```

```
        -- metadata bytes 0..3 = timestamp LE; byte 4 = rssi; byte 5 = snr.
        mv(31 downto 0)  := std_logic_vector(ts);
        mv(39 downto 32) := std_logic_vector(rssi);
        mv(47 downto 40) := std_logic_vector(snr);

        -- payload byte 2 = beacon; byte 6 = mod/pkt_type; byte 7 = SF;
        -- payload bytes 13,14 = pause target/duration.
        pv(23 downto 16)   := beacon;
        pv(55 downto 48)   := mod_b6;
        pv(63 downto 56)   := x"07";
        pv(111 downto 104) := tgt;
        pv(119 downto 112) := dur;

        m <= mv;
        p <= pv;
        fv <= '1';
        wait until rising_edge(clk);
        fv <= '0';
        wait until feat_busy = '0';
        wait until rising_edge(clk);
    end procedure;
begin
    clk <= not clk after CLK_PERIOD / 2;

    u_feat : entity work.feature_extract
        port map (clk => clk, rst_n => rst_n,
                  frame_valid => frame_valid,
                  metadata => metadata, payload => payload,
                  feat_busy => feat_busy,
                  bram_wr_en => wr_en, bram_wr_addr => wr_addr, bram_wr_data => wr_data,
                  bram_rd_addr => rd_a, bram_rd_data => rd_da);

    u_bram : entity work.feature_bram
        port map (clk => clk, rst_n => rst_n,
                  wr_en => wr_en, wr_addr => wr_addr, wr_data => wr_data,
                  rd_addr_a => rd_a, rd_data_a => rd_da,
                  rd_addr_b => rd_b, rd_data_b => rd_db);

    process
    begin
        wait for 100 ns;
        rst_n <= '1';
        wait for 200 ns;

        -- 3 DATA frames from ID 0x01: mod_code = 0x04 (mod_kind=1 LoRa, pkt_type=0).
        inject_frame(frame_valid, metadata, payload,
                     x"01", x"04", x"00000064", to_signed(-50, 8),
                     to_signed(20, 8), x"00", x"00");
        inject_frame(frame_valid, metadata, payload,
                     x"01", x"04", x"00000078", to_signed(-48, 8),
                     to_signed(21, 8), x"00", x"00");
```

```
        inject_frame(frame_valid, metadata, payload,
                     x"01", x"04", x"0000008C", to_signed(-49, 8),
                     to_signed(22, 8), x"00", x"00");

        -- 2 ACK frames from ID 0x02: mod_code = 0x05 (pkt_type=1).
        inject_frame(frame_valid, metadata, payload,
                     x"02", x"05", x"00000070", to_signed(-55, 8),
                     to_signed(18, 8), x"00", x"00");
        inject_frame(frame_valid, metadata, payload,
                     x"02", x"05", x"00000098", to_signed(-54, 8),
                     to_signed(19, 8), x"00", x"00");

        -- 1 PAUSE frame from ID 0xFF: mod_code = 0x06 (pkt_type=2),
           target=0x01, dur=0x05.
        inject_frame(frame_valid, metadata, payload,
                     x"FF", x"06", x"000000C0", to_signed(-30, 8),
                     to_signed(25, 8), x"01", x"05");

        -- Read back row 0x01 via host port and check.
        rd_b <= x"01";
        wait until rising_edge(clk);
        wait until rising_edge(clk);
        assert rd_db(95 downto 64) = x"00000003"
            report "row 0x01 frame_count expected 3" severity failure;
        assert rd_db(31 downto 24) = x"00"
            report "row 0x01 pkt_type expected 0x00 (DATA)" severity
               failure;

        rd_b <= x"02";
        wait until rising_edge(clk);
        wait until rising_edge(clk);
        assert rd_db(95 downto 64) = x"00000002"
            report "row 0x02 frame_count expected 2" severity failure;
        assert rd_db(31 downto 24) = x"01"
            report "row 0x02 pkt_type expected 0x01 (ACK)" severity
               failure;

        rd_b <= x"FF";
        wait until rising_edge(clk);
        wait until rising_edge(clk);
        assert rd_db(95 downto 64) = x"00000001"
            report "row 0xFF frame_count expected 1" severity failure;
        assert rd_db(31 downto 24) = x"02"
            report "row 0xFF pkt_type expected 0x02 (PAUSE)" severity
               failure;
        assert rd_db(231 downto 224) = x"01"
            report "row 0xFF target_id_if_pause expected 0x01" severity
               failure;
        assert rd_db(239 downto 232) = x"05"
            report "row 0xFF pause_duration_units expected 0x05" severity
               failure;

        report "tb_feature_extract PASS" severity note;
        std.env.finish;
```

```
    end process;
end sim;
```

Listing 4: `tb_feature_extract.vhd` — six-frame stimulus, three-row readback. Verifies frame counts, `pkt_type` field, and PAUSE-only fields.

Set `tb_feature_extract` as the simulation top (right-click in Sources → Simulation Sources → **Set as Top**), launch behavioral simulation, then in the Tcl console type `run 50 us` (once) and press Enter. Six injected frames plus three host-port readbacks keeps the test under 20 μs of simulated time; 50 μs gives margin. Confirm `tb_feature_extract PASS` prints in the Tcl console.

5 Verification Checklist

- ☐ `seg7_driver.vhd` deleted from project and disk.
- ☐ `feature_bram.vhd`, `feature_extract.vhd`, and updated `ingest_top.vhd` compile with zero errors.
- ☐ Post-synthesis utilization: BRAM = 2 tiles (the feature memory).
- ☐ `tb_feature_extract` prints `PASS`: row `0x01` count=3 type=0x00, row `0x02` count=2 type=0x01, row `0xFF` count=1 type=0x02 with target=`0x01` and duration=`0x05`.
- ☐ On hardware, the seven-segment display goes blank (all anodes high) but the LED activity continues.

Lab Handout 2-F Deliverable

1. `feature_bram.vhd`, `feature_extract.vhd`, and updated `ingest_top.vhd`.
2. `tb_feature_extract.vhd` and Tcl console transcript showing `PASS`.
3. Post-synthesis utilization screenshot showing BRAM = 2 tiles.

Glossary

BRAM	Block Random Access Memory	1, 3, 6, 9, 13
CRC	Cyclic Redundancy Check	1
FPGA	Field-Programmable Gate Array	1
RSSI	Received Signal Strength Indicator	1
RTL	Register-Transfer Level	1
SNR	Signal-to-Noise Ratio	1